

Script-Controlled ACL – Restrict Record Access Based on Field Value

Naan Mudhalvan / Skill Wallet Project Document

1. Project & Team Details

Metadata	Details
Project Title	Script-Controlled ACL – Restrict Record Access Based on Field Value
Team ID	SWTID-2026-9611
Team Leader	Adhsa Rani P
Team Members	Gayathri A, Parkavi P, Priyadharshini M, Chinthanika P
Platform / Environment	ServiceNow / SkillWallet (SmartBridge)

2. Problem Statement & Core Concept

Objective

Implement dynamic, record-level security within ServiceNow using Script-Controlled Access Control Lists (ACLs). The primary goal is to enforce security rules that evaluate field values (such as Department, State, Branch, or Sensitivity) in real time to grant or deny access to specific records.

Rationale

Static Role-Based Access Control (RBAC) evaluates user roles upfront but lacks context awareness regarding individual record data. Script-controlled ACLs bridge this gap by evaluating dynamic JavaScript conditions alongside explicit user roles during runtime, enabling contextual, fine-grained access governance.

3. Key Architecture & Operations

Security Target

- Target Table: `u_institution_details`

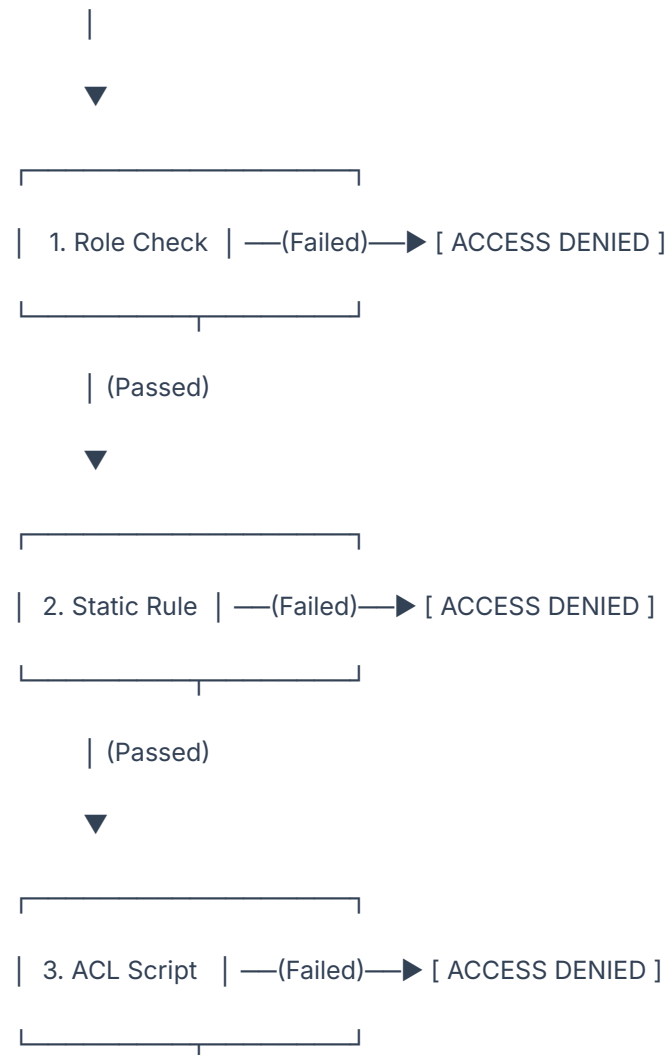
- **Operations Covered:** READ, WRITE, CREATE, DELETE

Evaluation Pipeline

When a user attempts to access or modify a record, ServiceNow processes the request through a multi-tiered evaluation pipeline. All steps must evaluate to **true** for access to be granted:

1. **Role Check:** Validates whether the requesting user possesses the designated security roles (e.g., custom roles `bb1`, `bb2`, `bb3`, `bb4`).
2. **Static Condition Builder:** Evaluates basic declarative filter conditions defined directly on the ACL form.
3. **Scripted Condition:** Executes context-aware JavaScript code to dynamically evaluate runtime record attributes (e.g., checking if `current.u_branch == 'EEE'`).

[Incoming User Request]



| (Passed)



[ACCESS GRANTED]

4. Implementation Specification

The following configuration demonstrates the Scripted ACL implementation for the **DELETE** operation on the target table.

Configuration Parameters

- Type: record
- Operation: Delete
- Name / Table: u_institution_details
- Active: true
- Requires Role: bb4

Script Logic Strategy

```
t
(function executeRule(current, previous) {
// 1. Verify required role explicit check
if (!gs.hasRole('bb4')) {
return false;
}
// 2. Dynamic Field Value Access Check

if (current.u_branch == 'EEE') {

return true;

}

// Default fallback deny

return false;

})();
```

5. Brainstorming & Use Cases Matrix

Use Case	Trigger / Condition	Access Rule Strategy	Target Roles
:---	:---	:---	:---

| ****1. Confidential Records**** | `is\_confidential == true` | Restrict read and edit rights exclusively to record owners and elevated security roles. | `bb3`, `bb4` |

| ****2. Lifecycle State Locking**** | `state IN ('Closed', 'Approved')` | Lock record modification by evaluating state; deny `WRITE` operations automatically. | All Roles (`bb1`–`bb4`) |

| ****3. Departmental Segregation**** | `u\_branch == user.department` | Match record branch/department directly with the logged-in user's profile attributes. | `bb1`, `bb2` |

6. Testing, Verification & Best Practices

Verification Matrix

| Role | Read | Create | Edit (Write) | Delete | Functional Constraints |

| :--- | :---: | :---: | :---: | :---: | :--- |

| **bb1** | Read Only | × | × | × | Restricted to non-confidential department records. |

| **bb2** | Read | Create | × | × | Can create new records within primary assigned branch. |

| **bb3** | Read | Create | Edit | × | Modify active records; restricted on locked states. |

| **bb4** | Read | Create | Edit | Delete | Full rights; Delete restricted to `u\_branch == 'EEE'`. |

Interface Consistency

To maintain a unified platform experience, ACL permissions are automatically enforced across all client interfaces:

* **Form Views:** Read-only fields and hidden buttons reflect ACL evaluation automatically.

* **List Views:** Unreadable or restricted records are filtered seamlessly without breaking pagination.

* **Service Portal:** Widget components and data queries adhere strictly to server-side ACL evaluations.

Performance Best Practices

1. **Lightweight Execution:** Avoid heavy synchronous operations or complex loops within the ACL script.

2. **Prevent Database Queries:** Avoid `GlideRecord` database lookups inside ACL scripts. Leverage cached user session properties (`gs.getUser()`) and inline `current` record object values instead.

3. ****Order of Evaluation:**** Maximize performance by placing low-cost static conditions and role checks prior to script execution.